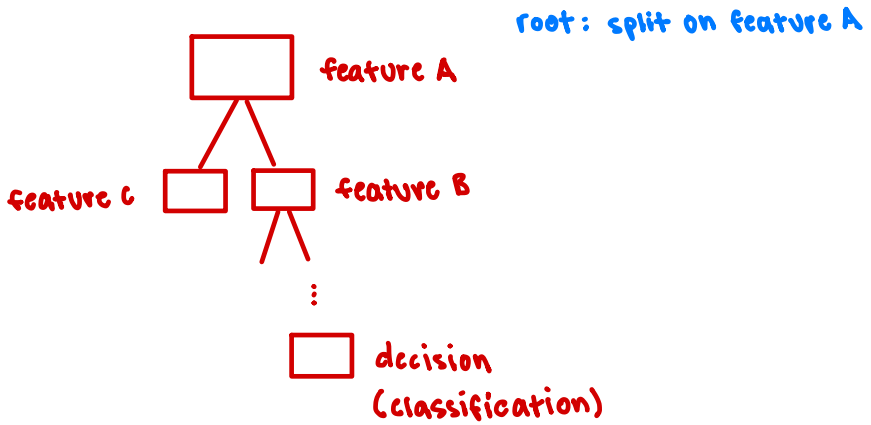
Week 6, Decision trees

CAPP 30254, Machine Learning for Public Policy

Decision trees

A *decision tree* is a machine learning model where a tree-like structure is created in order to make classification decisions.

For each node in the tree, the idea is to *split* the data into subgroups based on feature values, creating branches and child nodes. The last split creates the leaves of the tree, which make classification decisions.



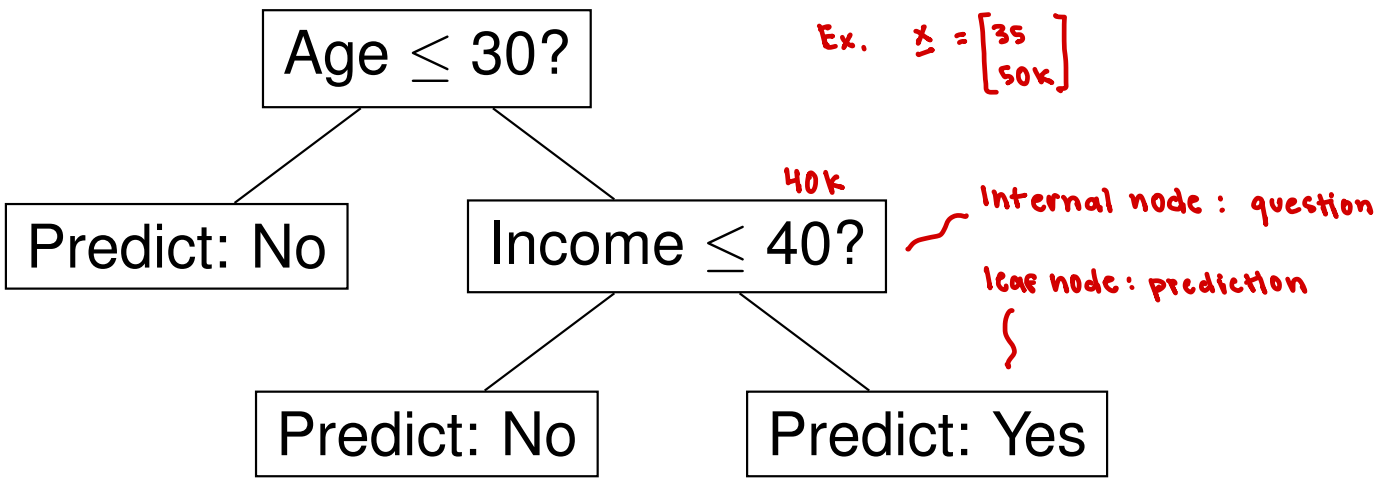
"Voting" decision tree

The tree is learned from the training data:
- Which features to use — feature selection
- What value to split at — threshold splitting

Consider the problem of predicting whether a person will vote based on their age and income. A decision tree for this problem might look like this:

data $\underline{x}$ has two features: age, income

Ex. $\underline{x} = \begin{bmatrix} 35 \\ 50k \end{bmatrix}$



To construct the internal nodes of a decision tree, a feature needs to be selected and split at some threshold.

Splitting by impurity

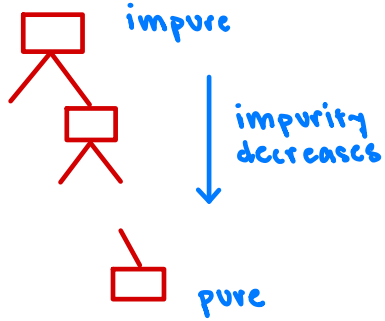
A set of labeled data is *pure* if every label is the same. If the leaves of a decision tree have pure data, we can confidently assign a label to new data.

Each split should reduce the impurity of the data, measured by an *impurity measure*, as much as possible.

- data starts highly impure
- want leaves to be pure

Common ways to measure impurity are:

- Gini impurity
- Entropy



Gini impurity

The *Gini impuriy* is a measure of how pure or mixed the labels in a dataset are. The Gini impurity is given by:

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

where C is the number of classes and p_i is the fraction of data of class i at that node.

Ex.

- 7 apples
- 2 oranges
- 1 peach

→ C = 3

$P_1 = 7/10$, $P_2 = 2/10$, $P_3 = 1/10$

$G = 1 - [(7/10)^2 + (2/10)^2 + (1/10)^2]$

$= 1 - [54/100]$

$= 0.46$

Entropy

Information gain:
difference between impurity at the parent node and a weighted average of impurity at the children

Information gain:
$H(\text{parent}) - [P_{\text{left}} H(\text{left}) + P_{\text{right}} H(\text{right})]$

proportion of data at left and right child node

Entropy is another way to measure how pure or mixed a dataset is. Entropy is calculated by:

$$H = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

For binary classification:

p: fraction of data in +1 class

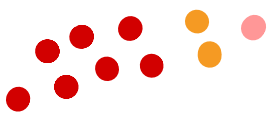
$H = -p \log_2 p - (1-p) \log_2 (1-p)$

$0 \leq p \leq 1$

- If all from same class $H = 0$
- If split 50/50 $H = 1$

where again C is the number of classes and p_i is the fraction of data of class i at that node.

Ex.



- 7 apples
- 2 oranges
- 1 peaches

→ $C = 3$

$p_1 = 7/10$

$p_2 = 2/10$

$p_3 = 1/10$

$H = -\left[7/10 \log_2(7/10) + 2/10 \log_2(2/10) + 1/10 \log_2(1/10) \right]$

$= 1.156$

Building a decision tree

Decision trees are constructed recursively by greedily picking the best feature at the current node and splitting on that feature to create branches.

- The root starts with all the labeled training data
- Choose the best feature and threshold to split on
- Split the training data into branches
- Repeat recursively keep splitting until a stopping condition is met
- At the leaves, assign the most commonly occurring class among the data as the prediction

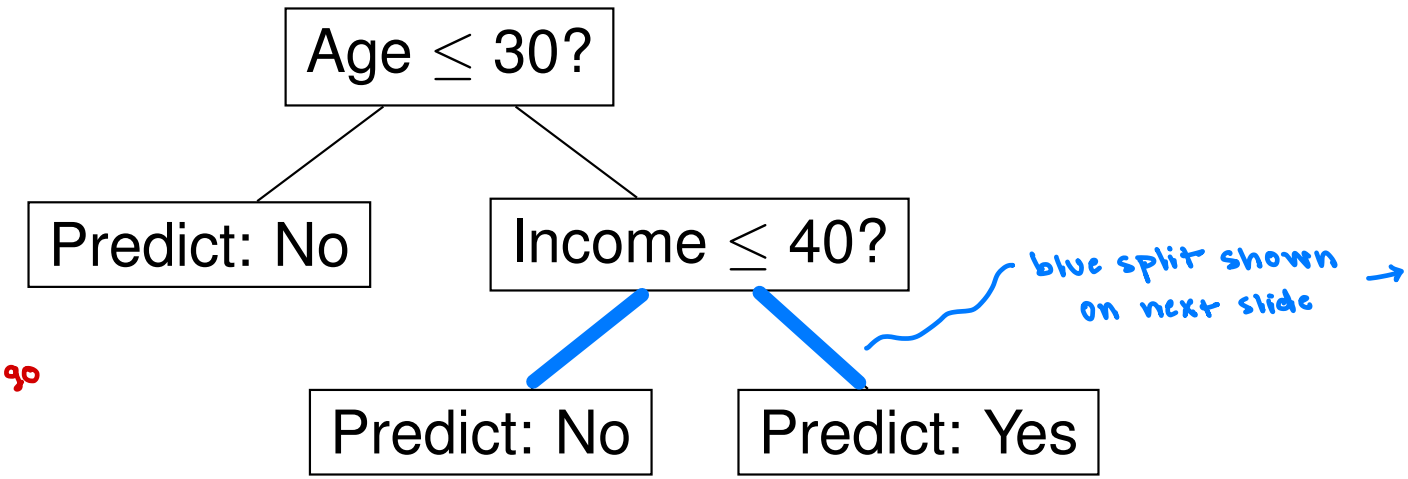
Making predictions

After a tree is constructed, it can be used to predict a label for new data. Consider again the voting tree:

Ex.

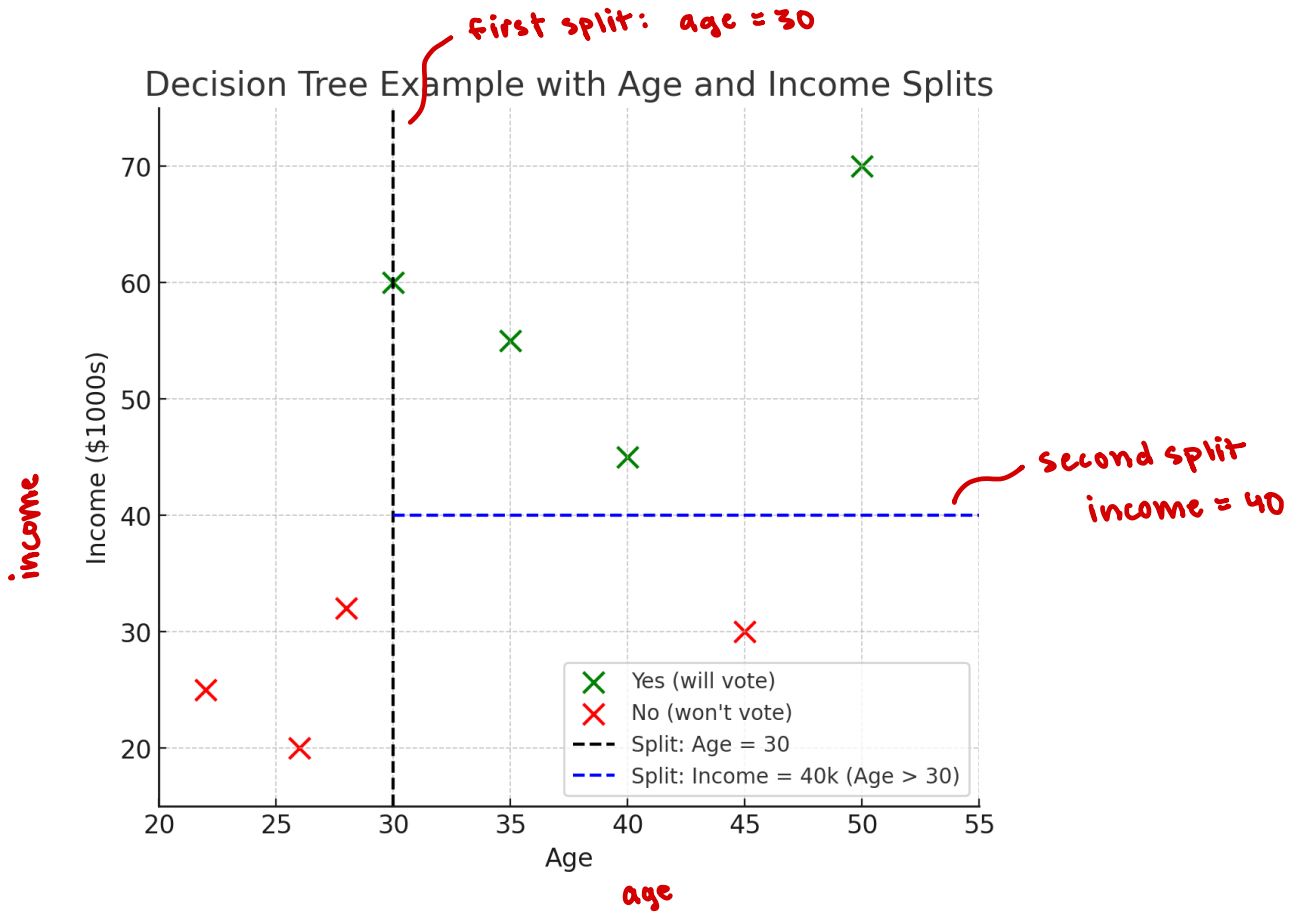
$\underline{x}$: age = 38
income = 20k

- start at root
- age $\leq$ 30? No, go left
- income $\leq$ 40? Yes, go right
- prediction: No



Decision boundaries

Decision trees can represent interesting decision boundaries. For example, here is the decision boundary for the simple voting decision tree:



Overfitting problem

could keep splitting until each sample has its own leaf node

Because the algorithm to create a decision tree calls for nodes to keep splitting as long as impurity improves, it's easy for decision trees to become overfit. Without limits, they'll keep going until they fit the training data perfectly.

Like other overfit models, overfit decision trees memorize the training data, but don't find underlying patterns in the data. They perform very well on the training data (low error), but generalize poorly to new, unseen data.

There are a number of strategies to prevent overfitting, including stopping conditions and pruning branches from a tree after it has been built.

Stopping conditions

To help with overfitting, common stopping conditions are:

- Don't allow a tree to grow deeper than some maximum number of levels Ex. no deeper than 10 levels
- Don't allow a tree to have fewer than some number of data points in a node Ex. every node has at least 5 nodes
- Stop when impurity improves by less than some small value ϵ

Pruning

How it works:
- grow the full tree
- evaluate if replacing a subtree w/ a leaf would maintain validation accuracy
- if yes, prune the subtree

try for branches in the tree

When some branches of a decision tree are determined to be unhelpful in predicting the target, they can be *pruned*, or removed. This can happen when:

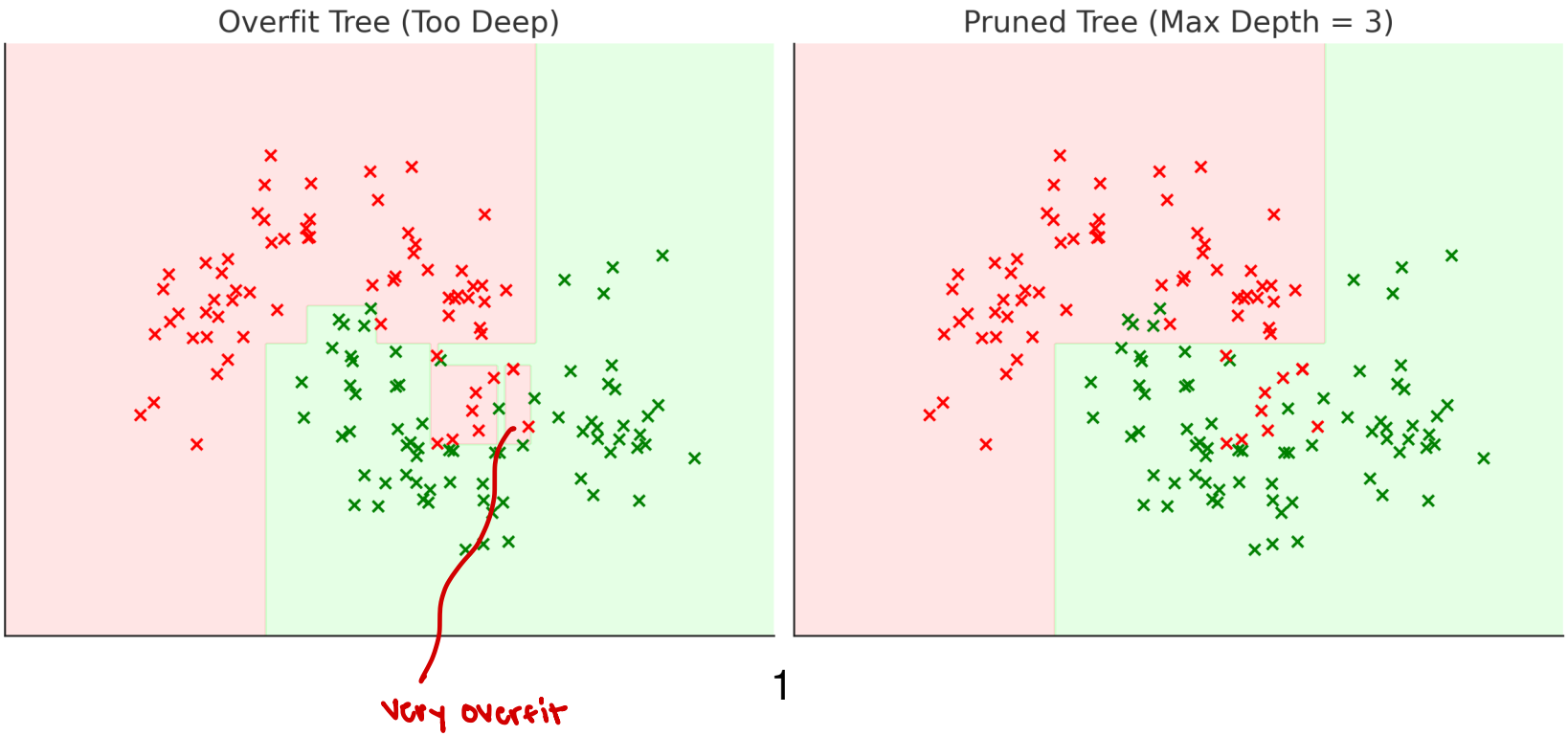
- A tree is too deep or too complex to perform well in general
- When splits in the tree decrease the impurity very little

Pruning simplifies the tree to help avoid overfitting and improve the generalization of the tree.

Pruning can be done to simplify the tree after growing the tree fully first. In this approach, branches are cut off if pruning them improves accuracy on the validation set.

Pruning

Here's a visual comparison of an overfit, fully grown tree and a pruned tree.



[1]

[1] Generated by ChatGPT with the prompt: "Could you show the decision boundary of an overfit vs. pruned decision tree?"

2

[2] Generated by GPT-4, DALL-E 3 after the prompt: "Hi, could you please make a minimalistic logo for a machine learning class..."